Fashion.45

Name	Kurup AnandhuKrishne Vinodkumar
Roll no.	2025BCY0013
Subject	ICS214 IT Workshop III
College	IIIT Kottayam
Submission Date	28/10/2026

Audited by AnandhuKrishne on 21:25 28/10/2026 referencing the project files and the readme.md file

About and Introduction:

Fashion.45 is a Python-based desktop e-com app that uses a GUI with Tkinter, a Flask REST API, a SQLite rdbms, and networkX graph analytics. It supports product browsing, category filtering, a shopping cart, checkout, show product images and bundling suggestions on its storefront and inventory management, order viewing on its admin interface.

The bundle feature models each order and each product as nodes on a connected bipartite graph which are connected with weighted projection; where edge weight is the no. of connections btw those nodes. Matplotlib is used for the visual representation of the graph.

How it approaches this?

Layer	What it does?	Library used and its files
API endpoints	It handles the backend of the application that is acts a bridge to talk between user sent instructions on frontend to system and process it.	Flask and Flask-CORS API; app.py
RDBMS	It handles the relations between different elements and also the database itself.	SQLite; database.py
Desktop Client	It acts as a bridge between pure code and user gestures to enable the user to talk to the computer using the visual elements like shopping cart, orders, name, etc.	Tkinter, Requests, Pillow, Matplotlib; desktop_app.py
Recommendation Engine	Builds a weighted projection graph and helps calculate the bundle recommendation	NetworkX and pandas; app.py itself as a function

Objectives:

Building a desktop app with product cards, category-based filtering, cart totals, and checkout

Serve a live dbms based Flask Rest API.

Use a normalized 3nf rdbms enforcing foreign key(`PRAGMA foreign_keys = ON;`) to gurantee referential consistency across categories.

Uses Recommendation engine to build a weighted recommendation graph and help in bundle recommendation.

How bipartite graph recommendations work?

Fashion.45 uses a bipartite graph to connect customer orders with the products purchased in them. The graph is represented as ($B=(U,V,E_B)$), where:

- ***U* (Orders):** Represents the different customer transactions.
- ***V* (Products):** Represents the products available in the catalog.
- **(E_B) (Edges):** Connects an order to a product if that product was purchased in that order.

For example, if Order 1 contains Product 1 and Product 2, both products are connected to Order 1. If Order 2 contains Product 1 and Product 3, the same process is repeated for that order.

This creates a relationship between orders and products, which can then be used to identify items that are frequently purchased together.

Implementation note: Node disambiguation

Both orders and products use integer IDs in SQLite, and both can start from 1. If these IDs are passed directly into NetworkX, an order and a product with the same ID could be treated as the same node.

To avoid this, Fashion.45 prefixes order nodes with `order_`, such as `order_1`, while product nodes retain their integer IDs. This ensures that orders and products remain separate sets of nodes.

Weighted one-mode projection

The bipartite graph is then converted into a weighted, undirected graph containing only product nodes.

In this projected graph, two products are connected if they have appeared together in at least one order. The edge weight represents the number of orders in which both products were purchased together.

For example, if Product 1 and Product 2 were purchased together in two different orders, the weight of their connecting edge would be 2. If Product 1 and Product 3 appeared together only once, their edge weight would be 1.

In simple terms, a higher edge weight means that the two products have been purchased together more frequently.

How the bundle recommendation works

When the user clicks the **Bundle** button on a product card, Fashion.45 checks the projected graph to find products connected to the selected product.

The recommendation process follows these steps:

1. Identifies the neighbouring products connected to the selected product.
2. Sorts these products according to their edge weights in descending order.
3. Selects the highest-weight neighbour as the primary co-purchase recommendation.
4. Displays the recommended product and its co-purchase frequency on the storefront.

For example, if a selected product has a connection weight of 5 with Product A and a weight of 2 with Product B, Product A is ranked higher because it has appeared alongside the selected product in more orders.

This allows Fashion.45 to generate bundle suggestions from existing order data instead of relying only on product categories or manually selected combinations.

One limitation is that the recommendation depends on previous orders. Products with no recorded co-purchases may not have enough connections to generate a useful bundle.

Database Schema and Data Modelling

Fashion.45 uses SQLite as its relational database management system. It stores the product catalog, categories, customer orders, and individual items within each order.

The database is divided into four main tables:

- **categories:** Stores the different product categories and their IDs.
- **products:** Stores product details such as product name, price, stock, category ID, and image URL.
- **orders:** Stores transaction details such as order ID, customer name, order date, and total amount.
- **order_items:** Stores the products included in each order, along with their quantities and related IDs.

The `order_items` table acts as a junction table between orders and products. This allows an order to contain multiple products and allows the same product to appear in multiple orders.

Foreign key enforcement is enabled using `PRAGMA foreign_keys = ON;` when establishing the database connection. This helps maintain referential integrity between related records.

Online product synchronisation

Fashion.45 also supports synchronisation with the FakeStore API through `database.py`.

The application sends a GET request to `https://fakestoreapi.com/products` to retrieve product information such as titles, prices, categories, and image URLs.

The retrieved data is then mapped to the local database, where existing products can be updated and new products can be inserted.

If the online API is unavailable, the application can initialise using its fallback catalog. This helps the application remain usable even when the external service cannot be reached.

REST API and Backend

The backend is developed using Flask and Flask-CORS in `app.py`. It acts as the communication layer between the desktop client and the database.

The desktop client sends HTTP requests to the backend whenever it needs to retrieve products, filter categories, place orders, manage inventory, or request recommendations.

The main API endpoints are:

Method	Endpoint	Purpose
GET	`/`	Checks API status and displays endpoint information.
GET	`/api/categories`	Retrieves the available categories.

GET	`/api/products`	Retrieves products, with optional category filtering.
POST	`/api/order`	Processes checkout and updates the database.
GET	`/api/recommendations/<id>`	Retrieves product bundle recommendations.
GET	`/api/graph-data`	Returns graph data for visualisation.
GET	`/api/analytics/summary`	Retrieves sales and inventory summary information.
GET	`/api/admin/orders`	Retrieves recent orders.
POST	`/api/admin/products`	Adds a product to the catalog.
PUT	`/api/admin/products/<id>`	Updates product information or stock.
DELETE	`/api/admin/products/<id>`	Deletes a product and handles related records.
POST	`/api/admin/sync-online`	Triggers online product synchronisation.

The API separates the user interface from database operations. This makes the application easier to organise and allows the desktop client to communicate with the backend through defined endpoints.

Desktop Client

The desktop client is implemented in `desktop_app.py` using CustomTkinter, Requests, Pillow, and Matplotlib.

It provides the visual interface through which the user interacts with Fashion.45. The interface includes product cards, category filtering, product images, a shopping cart, checkout, bundle recommendations, and administrative features.

Background processing

Network requests are handled using background threads so that the interface does not have to wait for every request to finish before responding to user actions.

The application uses `threading.Thread` to run these operations and `self.after()` to schedule interface updates on the main thread.

This helps prevent the interface from freezing when the backend takes time to respond.

Product image loading

Product images are retrieved from their image URLs and processed using Pillow before being displayed.

The application caches loaded images so that it does not need to retrieve the same image repeatedly. Transparent PNG images are composited onto a white background, and thumbnails are resized to 60 × 60 pixels.

If an image cannot be loaded because of a broken URL or network timeout, a placeholder icon is displayed instead.

Checkout and order processing

The user can add products to the cart, change quantities, and review the total before placing an order.

When the user clicks **Confirm Order (SQL Commit)**, a dialog allows the customer name to be confirmed or updated. The order is then sent to the Flask backend for validation and database processing.

The backend checks product availability and records the transaction while updating the corresponding inventory quantities.

Graph visualisation

Fashion.45 also includes a graph visualisation window to display the relationships between products.

The application retrieves graph data from `/api/graph-data`, uses NetworkX to construct the graph, and applies `nx.spring_layout()` to position the nodes.

Matplotlib is used to display the graph inside a separate desktop window. The graph window also handles its closing event to release the associated Matplotlib figure resources.

This makes it possible to inspect the product relationships used by the recommendation engine instead of treating the bundle feature as a black box.

Conclusion

Fashion.45 combines desktop application development, database management, REST API integration, and graph theory in a single e-commerce project.

The application supports product browsing, category filtering, cart management, checkout, inventory updates, order viewing, and product bundle recommendations.

SQLite is used to store and manage the data, while Flask provides the backend API. NetworkX is used to analyse the relationships between products, and Matplotlib provides a visual representation of the resulting graph.

The project also includes online product synchronisation, image caching, and background request processing to improve usability.

Overall, Fashion.45 demonstrates how concepts from ICS214: IT Workshop III can be applied to a practical application, particularly in the areas of relational database management, desktop development, API communication, and graph-based recommendation systems.